

Ответы к экзамену

Рекомендательные системы и коллаборативная фильтрация

Содержание

1 ПОЛНАЯ ВЕРСИЯ ОТВЕТОВ	2
2 Раздел 1. Введение и основы рекомендательных систем	2
2.1 1. История развития рекомендательных систем: Grundy, Tapestry, GroupLens, Netflix Prize	2
2.2 2. Классификация рекомендательных систем. Prediction и Ranking	3
3 Раздел 2. Простые и memory-based методы	3
3.1 3. Простые рекомендации на основе популярности. IMDB Top-N. User-Based и Item-Based CF	3
3.2 4. Memory-Based Collaborative Filtering	4
3.3 5. Метрики сходства: Cosine Similarity, Pearson Correlation и другие	4
3.4 6. kNN в рекомендательных системах. Библиотека Surprise	5
4 Раздел 3. Матричные методы и факторизация	5
4.1 7. Matrix Factorization и классический SVD	5
4.2 8. FunkSVD	6
4.3 9. Pointwise, Pairwise и Listwise подходы	6
4.4 10. Retrieval, Ranking и Re-ranking	7
5 Раздел 4. Контентная и гибридная фильтрация	7
5.1 11. Content-Based Filtering: профиль пользователя, профиль объекта, Utility Matrix	7
5.2 12. Bag-of-Words, TF-IDF и метаданные	7
5.3 13. Косинусная мера и content-based рекомендации	8
5.4 14. Гибридные рекомендательные системы	8
5.5 15. Мультимодальные данные: Text Embeddings и Image Embeddings	9
6 Раздел 5. LightFM и implicit feedback	9
6.1 16. LightFM как гибридная рекомендательная модель	9
6.2 17. Архитектура LightFM и embeddings признаков	9
6.3 18. Loss-функции LightFM: Logistic, BPR, WARP, k-OS WARP	10
6.4 19. OpenMP и многопоточность в LightFM	10
6.5 20. Implicit Feedback и Explicit Feedback	10
6.6 21. ALS: идея, постановка, sparse-матрицы	11
6.7 22. ALS, BPR, WARP и LightFM	11
7 Раздел 6. Нейросетевые рекомендательные системы	11
7.1 23. Neural Collaborative Filtering: GMF, MLP, NeuMF	11
7.2 24. AutoRec как encoder-decoder подход	12
8 Раздел 7. Последовательные и графовые рекомендательные системы	12
8.1 25. GRU4Rec, SASRec и BERT4Rec	12
8.2 26. Recall@K и MRR	13
8.3 27. DeepWalk, Node2Vec и ограничения random walks	13
8.4 28. Message Passing, Propagation и LightGCN	14
8.5 29. NGCF, GraphSAGE, GAT и PinSage	14
8.6 30. Attention в SASRec, BERT4Rec и GAT	15
9 КРАТКАЯ ВЕРСИЯ ДЛЯ ПОВТОРЕНИЯ	15
9.1 1. История рекомендательных систем	15

9.2	2. Классификация и постановки задач	15
9.3	3. Популярность, User-Based и Item-Based CF	15
9.4	4. Memory-Based CF	15
9.5	5. Метрики сходства	15
9.6	6. kNN и Surprise	16
9.7	7. Matrix Factorization и SVD	16
9.8	8. FunkSVD	16
9.9	9. Pointwise, Pairwise, Listwise	16
9.10	10. Retrieval, Ranking, Re-ranking	16
9.11	11. Content-Based Filtering	16
9.12	12. Bag-of-Words, TF-IDF, метаданные	16
9.13	13. Cosine Similarity в content-based	16
9.14	14. Гибридные системы	16
9.15	15. Мультимодальные данные	16
9.16	16. LightFM	17
9.17	17. Архитектура LightFM	17
9.18	18. Loss-функции LightFM	17
9.19	19. OpenMP в LightFM	17
9.20	20. Implicit Feedback	17
9.21	21. ALS	17
9.22	22. ALS, BPR, WARP, LightFM	17
9.23	23. NCF: GMF, MLP, NeuMF	17
9.24	24. AutoRec	17
9.25	25. GRU4Rec, SASRec, BERT4Rec	17
9.26	26. Recall@K и MRR	18
9.27	27. DeepWalk и Node2Vec	18
9.28	28. Message Passing и LightGCN	18
9.29	29. NGCF, GraphSAGE, GAT, PinSage	18
9.30	30. Attention в рекомендациях	18

10 САМЫЙ КОРОТКИЙ ИТОГ

18

1 ПОЛНАЯ ВЕРСИЯ ОТВЕТОВ

2 Раздел 1. Введение и основы рекомендательных систем

2.1 1. История развития рекомендательных систем: Grundy, Tapestry, GroupLens, Netflix Prize

Рекомендательные системы появились из практической проблемы: объектов становится слишком много, и пользователю трудно выбрать нужный фильм, книгу, товар, песню или статью. Сначала рекомендации были похожи на экспертные правила, а затем стали строиться на данных о поведении пользователей.

Grundy - одна из ранних рекомендательных систем для книг. Она относил пользователя к некоторому типу или стереотипу и рекомендовала книги, подходящие этому типу. Например, если пользователь интересуется приключениями и наукой, система выбирает книги из соответствующих категорий. Это был экспертный подход: знания о книгах и типах пользователей задавались заранее.

Tapestry стала важной системой, потому что с ней связывают идею collaborative filtering. Пользователи могли оставлять оценки и аннотации к документам, а другие пользователи использовали эту информацию для фильтрации. Здесь впервые явно появилась мысль: люди могут помогать друг другу находить полезную информацию.

GroupLens развила идею коллаборативной фильтрации для рекомендаций новостей. Система смотрела, какие пользователи похожи по прошлым оценкам, и рекомендовала новости на основе поведения похожих пользователей. Это уже близко к современному user-based collaborative filtering.

Netflix Prize стал крупным этапом развития рекомендательных систем. Netflix предложил участникам улучшить качество предсказания оценок фильмов. Этот конкурс сильно популяризировал матричную факторизацию, ансамбли моделей и работу с большими sparse-матрицами.

Общая эволюция выглядит так:

экспертные правила -> стереотипы пользователей -> collaborative filtering -> матричная факторизация -> гибридные модели -> нейросетевые и графовые модели
Главный переход: от ручных правил к интеллектуальным моделям, которые сами учатся на данных.

2.2 2. Классификация рекомендательных систем. Prediction и Ranking

Рекомендательные системы можно классифицировать по источнику информации.

Content-Based Filtering рекомендует объекты, похожие на те, которые пользователь уже любил. Например, если пользователь смотрел фантастику, система советует другие фантастические фильмы.

Collaborative Filtering использует поведение других пользователей. Если два пользователя похожи по прошлым оценкам, одному можно рекомендовать то, что понравилось другому.

Hybrid Systems объединяют несколько подходов, например collaborative filtering и content-based filtering. Это позволяет использовать и историю взаимодействий, и признаки объектов.

Knowledge-Based системы используют правила и ограничения. Они полезны там, где пользователь редко совершает выбор: недвижимость, кредиты, автомобили, страховки.

Context-Aware системы учитывают контекст: время, место, устройство, сезон, текущую сессию.

Есть две основные постановки задачи. Prediction - предсказать оценку пользователя для объекта. Ranking - отсортировать объекты и показать Top-N.

Prediction: user u + item i -> predicted rating $\hat{r}_{ui}$

Ranking: user u -> ordered list of items $[i_1, i_2, \dots, i_K]$

Для prediction часто используют RMSE и MAE:

$$RMSE = \sqrt{\frac{1}{n} \sum_i (r_i - \hat{r}_i)^2}$$
$$MAE = \frac{1}{n} \sum_i |r_i - \hat{r}_i|$$

Для ranking используют Precision@K, Recall@K, NDCG@K, MRR. В реальных системах ranking обычно важнее, потому что пользователь видит именно отсортированный список рекомендаций.

3 Раздел 2. Простые и memory-based методы

3.1 3. Простые рекомендации на основе популярности. IMDB Top-N. User-Based и Item-Based CF

Самая простая рекомендательная система - показывать самые популярные объекты. Например: самые просматриваемые фильмы, самые покупаемые товары или самые читаемые статьи. Такой подход не персонализирован: все пользователи получают примерно одинаковые рекомендации.

В рейтингах типа IMDB нельзя просто сортировать фильмы по средней оценке. Фильм с одной оценкой 10/10 не должен быть выше фильма с тысячами оценок и средним рейтингом 8.8. Поэтому используют взвешенный рейтинг:

$$WR = \frac{v}{v+m}R + \frac{m}{v+m}C$$

где R - средняя оценка объекта, v - число голосов, m - минимальное число голосов для доверия, C - средняя оценка по всем объектам. Если голосов мало, рейтинг подтягивается к среднему C . Если голосов много, больше доверяем собственному рейтингу объекта.

Плюсы популярностных рекомендаций: простота, скорость, понятность, полезность для новых пользователей. Минусы: нет персонализации, новые и нишевые объекты плохо продвигаются.

User-Based Collaborative Filtering ищет похожих пользователей. Если пользователь A похож на пользователя B , то A можно рекомендовать то, что понравилось B .

$$\hat{r}_{ui} = \frac{\sum_{v \in N(u)} \text{sim}(u, v) r_{vi}}{\sum_{v \in N(u)} |\text{sim}(u, v)|}$$

Item-Based Collaborative Filtering ищет похожие объекты. Если пользователю понравился один объект, ему рекомендуются похожие на него объекты.

$$\hat{r}_{ui} = \frac{\sum_{j \in N(i)} \text{sim}(i, j) r_{uj}}{\sum_{j \in N(i)} |\text{sim}(i, j)|}$$

Item-Based CF часто стабильнее, потому что сходство объектов меняется медленнее, чем вкусы пользователей.

3.2 4. Memory-Based Collaborative Filtering

Memory-Based Collaborative Filtering - это методы, которые используют исходную user-item матрицу напрямую. Они не строят сложную параметрическую модель, а опираются на память о прошлых оценках или взаимодействиях.

User-item матрица выглядит так: строки - пользователи, столбцы - объекты, значения - оценки или факты взаимодействия. Большая часть ячеек обычно пустая, потому что каждый пользователь взаимодействует только с малой частью объектов.

Общий алгоритм memory-based CF:

1. Берем пользователя или объект.
2. Находим похожих пользователей или похожие объекты.
3. Смотрим их оценки/взаимодействия.
4. Предсказываем оценку или формируем Top-N.

Преимущества: простота, понятность, хорошая интерпретируемость. Можно объяснить: «мы рекомендуем этот фильм, потому что он понравился похожим пользователям».

Ограничения: разреженность данных, холодный старт, плохая масштабируемость на миллионы пользователей, популярностное смещение. При очень больших данных считать похожесть между всеми пользователями или объектами дорого.

3.3 5. Метрики сходства: Cosine Similarity, Pearson Correlation и другие

Метрики сходства нужны, чтобы понять, насколько похожи пользователи или объекты.

Cosine Similarity измеряет угол между двумя векторами:

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

Если два вектора направлены одинаково, сходство близко к 1. В рекомендациях это значит: пользователи имеют похожие предпочтения или объекты имеют похожие признаки.

Pearson Correlation учитывает отклонение оценок от среднего значения пользователя:

$$\text{sim}(u, v) = \frac{\sum_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_i (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_i (r_{vi} - \bar{r}_v)^2}}$$

Pearson полезен, потому что пользователи по-разному ставят оценки: один ставит почти всем 5, другой редко ставит выше 3. Корреляция сравнивает не абсолютные оценки, а относительные предпочтения.

Jaccard Similarity используется для множеств, особенно при implicit feedback:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Например, если два пользователя смотрели много одних и тех же фильмов, их Jaccard similarity будет выше.

Также используют MSD similarity, adjusted cosine similarity, Euclidean distance и другие меры. Выбор зависит от типа данных: ratings, clicks, purchases, views.

3.4 6. kNN в рекомендательных системах. Библиотека Surprise

k-Nearest Neighbors в рекомендациях означает: для пользователя или объекта нужно найти k ближайших соседей и использовать их для предсказания.

В user-based kNN мы ищем k похожих пользователей и усредняем их оценки с весами по сходству:

$$\hat{r}_{ui} = \frac{\sum_{v \in N_k(u)} sim(u, v) r_{vi}}{\sum_{v \in N_k(u)} |sim(u, v)|}$$

В item-based kNN мы ищем k похожих объектов:

$$\hat{r}_{ui} = \frac{\sum_{j \in N_k(i)} sim(i, j) r_{uj}}{\sum_{j \in N_k(i)} |sim(i, j)|}$$

Библиотека Surprise - популярная Python-библиотека для классических рекомендательных систем с explicit feedback. Она предоставляет модели KNNBasic, KNNWithMeans, KNNWithZScore, KNNBaseline, SVD, SVD++, NMF и другие.

Типичный pipeline Surprise:

1. Подготовить DataFrame: userId, itemId, rating.
2. Задать Reader с диапазоном рейтингов.
3. Загрузить Dataset.
4. Разделить train/test.
5. Обучить модель.
6. Получить predictions.
7. Посчитать RMSE или MAE.

KNNWithMeans лучше простого KNNBasic, потому что учитывает среднюю оценку пользователя или объекта. KNNBaseline дополнительно учитывает bias пользователя и объекта.

4 Раздел 3. Матричные методы и факторизация

4.1 7. Matrix Factorization и классический SVD

Матричная факторизация - это подход, при котором большую user-item матрицу R раскладывают на две меньшие матрицы: пользователей и объектов.

$$R \approx PQ^T$$
$$\hat{r}_{ui} = p_u^T q_i$$

Каждый пользователь получает латентный вектор p_u , каждый объект - латентный вектор q_i . Латентные факторы - это скрытые признаки, которые модель сама находит. Например, для фильмов это может быть «фантастика», «романтика», «комедия», «сложный сюжет», но в неявной форме.

Классическое сингулярное разложение SVD:

$$R = U \Sigma V^T$$

Если оставить только k главных факторов, получится приближенное восстановление матрицы:

$$R_k = U_k \Sigma_k V_k^T$$

Главное ограничение классического SVD: оно требует заполненную матрицу. В рекомендациях матрица почти всегда разреженная. Нельзя просто заполнить пропуски нулями, потому что ноль означает не «пользователю не понравилось», а «нет информации».

Также классический SVD плохо решает холодный старт и не оптимизирует напрямую ranking-метрики.

4.2 8. FunkSVD

FunkSVD - практический вариант матричной факторизации, ставший популярным во время Netflix Prize. В отличие от классического SVD, он обучается только на известных рейтингах и не требует заполненной матрицы.

Основная формула:

$$\hat{r}_{ui} = p_u^T q_i$$

Чаще используют расширенную формулу с bias:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$$

где μ - средняя оценка по системе, b_u - смещение пользователя, b_i - смещение объекта. Bias нужен, потому что некоторые пользователи обычно ставят оценки выше, а некоторые объекты в среднем оцениваются лучше.

Функция ошибки:

$$\min \sum_{(u,i) \in K} (r_{ui} - \hat{r}_{ui})^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

Обучение обычно идет через SGD. Ошибка:

$$e_{ui} = r_{ui} - \hat{r}_{ui}$$

Обновления:

$$\begin{aligned} p_u &\leftarrow p_u + \gamma(e_{ui}q_i - \lambda p_u) \\ q_i &\leftarrow q_i + \gamma(e_{ui}p_u - \lambda q_i) \end{aligned}$$

Главное отличие FunkSVD: это оптимизационная модель для sparse-данных, а не прямое математическое разложение полной матрицы.

4.3 9. Pointwise, Pairwise и Listwise подходы

В ranking-задачах модель должна не просто предсказать оценку, а правильно отсортировать объекты.

Pointwise рассматривает каждый объект отдельно. Модель предсказывает score, рейтинг или вероятность клика:

$$\hat{y}_{ui} = f(u, i)$$

Для рейтингов используют MSE, для кликов - binary cross-entropy. Минус: модель не обучается напрямую порядку объектов.

Pairwise рассматривает пары объектов. Для пользователя позитивный объект должен быть выше негативного:

$$\hat{r}_{ui} > \hat{r}_{uj}$$

Типичный пример - BPR loss:

$$L = -\log \sigma(\hat{r}_{ui} - \hat{r}_{uj}) + \lambda \|\Theta\|^2$$

Pairwise лучше подходит для Top-N рекомендаций, потому что оптимизирует относительный порядок объектов.

Listwise рассматривает весь список объектов сразу и пытается улучшать качество ранжированного списка целиком. Такие методы ближе всего к реальной выдаче, но сложнее в обучении.

Простая аналогия:

Pointwise: оцени каждый фильм отдельно.

Pairwise: сравни два фильма.

Listwise: отсортируй весь список.

4.4 10. Retrieval, Ranking и Re-ranking

В современных системах нельзя сразу точно отсортировать миллионы объектов для каждого пользователя. Поэтому используется многоэтапная архитектура.

Retrieval - быстрый отбор кандидатов. Из миллионов объектов выбирается несколько сотен или тысяч потенциально подходящих. На этом этапе важна скорость. Используются популярные объекты, nearest neighbors, matrix factorization, two-tower models, ANN search, graph retrieval.

Ranking - точная сортировка кандидатов. Модель получает меньше объектов, поэтому может использовать больше признаков: историю пользователя, признаки товара, контекст, цену, свежесть, вероятность клика или покупки.

Re-ranking - финальная корректировка выдачи. Здесь учитывают разнообразие, отсутствие дублей, бизнес-правила, безопасность, доступность товара, fairness.

Схема:

миллионы объектов -> Retrieval -> тысячи кандидатов -> Ranking -> сотни ->
↪ Re-ranking -> Top-N

Retrieval отвечает за масштаб, Ranking - за точность, Re-ranking - за качество финальной выдачи для пользователя и бизнеса.

5 Раздел 4. Контентная и гибридная фильтрация

5.1 11. Content-Based Filtering: профиль пользователя, профиль объекта, Utility Matrix

Content-Based Filtering рекомендует объекты, похожие на те, что пользователь уже любил. Система опирается на признаки объектов: жанр, теги, описание, категорию, цену, бренд, актёров и так далее.

Профиль объекта - это вектор признаков объекта. Например, фильм можно описать жанрами, актёрами, режиссёром, ключевыми словами и годом выпуска.

$x_i = [\text{genre_fantasy}, \text{genre_drama}, \text{keyword_space}, \text{keyword_family}, \dots]$

Профиль пользователя строится по объектам, которые ему понравились. Например, можно взять взвешенное среднее векторов понравившихся объектов:

$$p_u = \frac{\sum_{i \in I_u} r_{ui} x_i}{\sum_{i \in I_u} r_{ui}}$$

Utility Matrix - матрица полезности, где строки - пользователи, столбцы - объекты, значения - оценки или взаимодействия. В content-based подходе она помогает понять, какие объекты пользователь уже любил.

Рекомендация строится так:

$$\text{score}(u, i) = \text{sim}(p_u, x_i)$$

Плюсы: можно рекомендовать новые объекты, если у них есть признаки; рекомендации легко объяснить. Минусы: нужны хорошие признаки, есть холодный старт для нового пользователя, рекомендации могут быть однообразными.

5.2 12. Bag-of-Words, TF-IDF и метаданные

Объекты часто описываются текстом: название, описание, отзывы, теги. Чтобы использовать текст в модели, его нужно превратить в числовой вектор.

Bag-of-Words представляет текст как набор слов и их частоты. Порядок слов не учитывается.

Документ: "космос планета космос"

Словарь: [космос, планета, спорт]

Вектор: [2, 1, 0]

Минус Bag-of-Words: он не понимает смысл и порядок слов.

TF-IDF учитывает не только частоту слова в документе, но и редкость слова во всей коллекции:

$$TF\text{-}IDF(t, d) = TF(t, d) \cdot IDF(t)$$
$$IDF(t) = \log \frac{N}{DF(t)}$$

Слова, которые часто встречаются в конкретном документе, но редко встречаются в коллекции, получают большой вес. Например, «космос» информативнее, чем «фильм».

Метаданные - дополнительные признаки объекта: жанр, бренд, цена, год, категория, страна, актёры. Их можно кодировать через one-hot, multi-hot, числовые признаки или embeddings.

5.3 13. Косинусная мера и content-based рекомендации

Cosine similarity часто используется в content-based filtering для сравнения профиля пользователя и профиля объекта.

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

Если значение близко к 1, векторы похожи. Если близко к 0, сходства почти нет.

Алгоритм:

1. Представить объекты векторами.
2. Построить профиль пользователя.
3. Посчитать cosine similarity между пользователем и каждым объектом.
4. Отсортировать объекты по score.
5. Показать Top-N.

Например, если профиль пользователя содержит признаки «фантастика» и «космос», фильмы с такими же признаками получат высокий score.

Косинусная мера удобна для TF-IDF, потому что она сравнивает направление векторов, а не просто их длину.

5.4 14. Гибридные рекомендательные системы

Гибридные системы объединяют несколько подходов. Причина появления проста: у каждого отдельного метода есть слабые стороны. Collaborative filtering страдает от холодного старта, content-based filtering может быть однообразным. Гибридная система использует преимущества обоих.

Weighted Hybrid объединяет scores нескольких моделей:

$$score(u, i) = \alpha score_{CF}(u, i) + (1 - \alpha) score_{CB}(u, i)$$

Switching Hybrid переключается между методами. Например, для нового пользователя используем популярные или content-based рекомендации, а для пользователя с историей - collaborative filtering.

Mixed Hybrid показывает рекомендации из разных источников одновременно: «похожие товары», «популярное среди похожих пользователей», «новинки».

Feature Combination объединяет признаки в одной модели: ID пользователя, ID товара, категорию, цену, текст, историю поведения.

Cascade Hybrid использует одну модель для отбора кандидатов, а другую для уточнения порядка. Это похоже на Retrieval -> Ranking.

Гибридные системы обычно точнее, лучше работают с холодным стартом, но сложнее в реализации и интерпретации.

5.5 15. Мультимодальные данные: Text Embeddings и Image Embeddings

Мультимодальные данные - это данные разных типов: текст, изображения, видео, аудио, метаданные, поведение пользователя. В рекомендациях это особенно важно для товаров, фильмов, музыки и видео.

Text Embeddings - векторные представления текста. Текстовое описание объекта превращается в числовой вектор, который отражает смысл. Такие embeddings можно получать с помощью Word2Vec, FastText, BERT, Sentence-BERT и других моделей.

Image Embeddings - векторные представления изображений. Изображение пропускается через CNN или Vision Transformer и превращается в вектор:

$$v_i = CNN(image_i)$$

Для товара можно объединить текст, изображение и метаданные:

$$z_i = [text_embedding_i; image_embedding_i; metadata_i]$$

Мультимодальные модели помогают при холодном старте новых объектов и лучше понимают объект. Минусы: больше данных, больше вычислений, сложнее интерпретация.

6 Раздел 5. LightFM и implicit feedback

6.1 16. LightFM как гибридная рекомендательная модель

LightFM - гибридная модель, которая объединяет матричную факторизацию и признаки пользователей/объектов. Она может работать как collaborative filtering, если используются только ID, и как гибридная модель, если добавить признаки.

Основная формула:

$$\hat{r}_{ui} = p_u^T q_i + b_u + b_i$$

где p_u - embedding пользователя, q_i - embedding объекта, b_u и b_i - bias. Если score высокий, объект подходит пользователю.

LightFM полезен, когда есть взаимодействия и дополнительные признаки: жанры фильмов, категории товаров, теги, возраст пользователя, город, интересы.

Главный плюс LightFM - лучшая работа с холодным стартом. Если новый объект ещё никто не оценил, но у него есть признаки, модель может построить его embedding через признаки.

6.2 17. Архитектура LightFM и embeddings признаков

В LightFM embedding строится не только для пользователя или объекта, но и для их признаков.

Пользователь может иметь признаки: ID пользователя, город, интересы, сегмент. Итоговый embedding пользователя - сумма embeddings его признаков:

$$p_u = \sum_{f \in F_u} e_f$$

Объект может иметь признаки: ID объекта, жанр, категория, бренд, теги. Итоговый embedding объекта:

$$q_i = \sum_{g \in F_i} e_g$$

Итоговый score:

$$\hat{r}_{ui} = \left(\sum_{f \in F_u} e_f \right)^T \left(\sum_{g \in F_i} e_g \right) + b_u + b_i$$

Смысл: модель учит векторы признаков. Если признаки пользователя и объекта хорошо совпадают, score будет высоким.

6.3 18. Loss-функции LightFM: Logistic, BPR, WARP, k-OS WARP

LightFM поддерживает разные функции потерь под разные задачи.

Logistic Loss подходит для бинарной классификации: было взаимодействие или не было.

$$L = \log(1 + \exp(-y_{ui}\hat{r}_{ui}))$$

BPR - pairwise loss. Для пользователя позитивный объект должен быть выше негативного:

$$L = -\log \sigma(\hat{r}_{ui} - \hat{r}_{uj})$$

BPR хорошо подходит для implicit feedback и Top-N рекомендаций.

WARP - Weighted Approximate-Rank Pairwise. Он особенно старается улучшить верх выдачи. Если позитивный объект оказался далеко в списке, штраф большой. WARP сэмплирует негативные объекты до нахождения нарушения порядка.

k-OS WARP - разновидность WARP, которая использует более устойчивую оценку ранга через несколько негативных примеров.

Кратко:

Logistic - классификация взаимодействия.

BPR - pairwise ranking.

WARP - улучшение Top-K.

k-OS WARP - более стабильный Top-K ranking.

6.4 19. OpenMP и многопоточность в LightFM

LightFM использует OpenMP для многопоточного обучения на CPU. Это значит, что работа распределяется между несколькими ядрами процессора.

В коде это задается параметром:

```
model.fit(interactions, epochs=10, num_threads=4)
```

Многопоточность особенно полезна при большом числе пользователей, объектов и взаимодействий. WARP и BPR могут быть достаточно дорогими, поэтому ускорение важно.

Но ускорение не всегда линейное. 8 потоков не обязательно в 2 раза быстрее, чем 4, из-за накладных расходов, конкуренции за память и конфликтов при обновлении параметров.

Также результаты при многопоточном обучении могут немного отличаться между запусками, потому что обновления идут в разном порядке.

6.5 20. Implicit Feedback и Explicit Feedback

Explicit Feedback - явная обратная связь: рейтинг, лайк, дизлайк, отзыв, оценка звёздами. Пользователь прямо говорит, понравился объект или нет.

Implicit Feedback - неявная обратная связь: клик, просмотр, покупка, добавление в корзину, время просмотра, повторное прослушивание. Пользователь не говорит прямо, но его действие является сигналом интереса.

Главное отличие: отсутствие взаимодействия не означает негатив. Если пользователь не купил товар, это может значить, что он его просто не видел.

Для implicit feedback часто разделяют preference и confidence:

$$p_{ui} = \begin{cases} 1, & r_{ui} > 0 \\ 0, & r_{ui} = 0 \end{cases}$$
$$c_{ui} = 1 + \alpha r_{ui}$$

Preference показывает наличие интереса, confidence показывает уверенность в сигнале. Например, 10 просмотров - более сильный сигнал, чем 1 случайный клик.

Плюс implicit feedback: его много и он легко собирается. Минус: он шумный и неоднозначный.

6.6 21. ALS: идея, постановка, sparse-матрицы

ALS - Alternating Least Squares, метод матричной факторизации. Он раскладывает матрицу взаимодействий на матрицу пользователей X и матрицу объектов Y :

$$R \approx XY^T$$
$$\hat{r}_{ui} = x_u^T y_i$$

ALS называется alternating, потому что обучается попеременно:

1. Фиксируем item-векторы Y , обновляем user-векторы X .
2. Фиксируем user-векторы X , обновляем item-векторы Y .
3. Повторяем.

Для explicit feedback минимизируется ошибка по известным рейтингам:

$$\min_{X,Y} \sum_{(u,i) \in K} (r_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right)$$

Для implicit feedback используется preference и confidence:

$$\min_{X,Y} \sum_{u,i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right)$$

ALS хорошо работает с большими sparse-матрицами. Sparse-форматы CSR, CSC, COO хранят только ненулевые взаимодействия, а не всю огромную таблицу.

6.7 22. ALS, BPR, WARP и LightFM

ALS - матричная факторизация для больших разреженных матриц, особенно implicit feedback. Он хорошо масштабируется, но не оптимизирует ranking-метрики напрямую.

BPR - pairwise ranking. Он учит модель ставить позитивный объект выше негативного. Хорошо подходит для Top-N рекомендаций.

WARP - ranking loss, ориентированный на верх выдачи. Он полезен, когда важны первые позиции рекомендаций.

LightFM - гибридная модель, которая может использовать ID и признаки пользователей/объектов. Она может обучаться с Logistic, BPR, WARP и k-OS WARP.

Сравнение:

ALS - восстановление матрицы, хорошо для больших implicit sparse-данных.
BPR - pairwise ранжирование.
WARP - оптимизация верхних позиций Top-K.
LightFM - гибридная факторизационная модель с признаками.

Если есть только большая матрица взаимодействий - можно брать ALS. Если важен порядок - BPR или WARP. Если есть признаки - LightFM.

7 Раздел 6. Нейросетевые рекомендательные системы

7.1 23. Neural Collaborative Filtering: GMF, MLP, NeuMF

NCF - Neural Collaborative Filtering, нейросетевой подход к collaborative filtering. В классической matrix factorization интерес считается скалярным произведением p_u и q_i . NCF заменяет это более гибкой нейросетевой функцией.

GMF - Generalized Matrix Factorization. Она использует поэлементное произведение embeddings пользователя и объекта:

$$h_{ui} = p_u \odot q_i$$
$$\hat{r}_{ui} = \sigma(h^T(p_u \odot q_i))$$

GMF похожа на обычную матричную факторизацию, но более гибкая.

MLP - Multi-Layer Perceptron. Embeddings пользователя и объекта объединяются:

$$z = [p_u; q_i]$$

Затем проходят через несколько полносвязных слоев. MLP может находить сложные нелинейные зависимости.

NeuMF - Neural Matrix Factorization. Она объединяет GMF и MLP:

$$\begin{aligned}\phi_{GMF} &= p_u^G \odot q_i^G \\ \phi_{MLP} &= MLP([p_u^M; q_i^M]) \\ \hat{r}_{ui} &= \sigma(h^T[\phi_{GMF}; \phi_{MLP}])\end{aligned}$$

Смысл: GMF ловит простое совпадение скрытых факторов, MLP - сложные зависимости, NeuMF использует оба источника информации.

7.2 24. AutoRec как encoder-decoder подход

AutoRec - рекомендательная модель на основе автоэнкодера. Автоэнкодер получает вектор оценок и пытается восстановить его, включая пропущенные значения.

Схема:

ratings vector -> encoder -> hidden representation -> decoder -> reconstructed
↪ ratings

Encoder сжимает входной вектор:

$$h = f(Wr + b)$$

Decoder восстанавливает его:

$$\hat{r} = g(Vh + c)$$

User-Based AutoRec получает на вход вектор оценок одного пользователя и восстанавливает его. Item-Based AutoRec получает на вход вектор оценок одного объекта всеми пользователями. Ошибка считается только по известным рейтингам:

$$L = \sum_{i \in K} (r_i - \hat{r}_i)^2 + \lambda \|\Theta\|^2$$

AutoRec можно понимать как нейросетевой способ восстановить пропущенные оценки. Плюсы: умеет моделировать нелинейности. Минусы: нужна достаточно большая матрица, есть проблема разреженности.

8 Раздел 7. Последовательные и графовые рекомендательные системы

8.1 25. GRU4Rec, SASRec и BERT4Rec

Последовательные рекомендательные системы учитывают порядок действий пользователя. Задача: по истории действий предсказать следующий объект.

$$S_u = [i_1, i_2, \dots, i_t] \rightarrow i_{t+1}$$

GRU4Rec использует рекуррентную сеть GRU. Она обновляет скрытое состояние после каждого действия:

$$h_t = GRU(x_t, h_{t-1})$$

GRU4Rec хорошо подходит для session-based рекомендаций, но RNN хуже параллелятся и могут хуже работать с длинными зависимостями.

SASRec использует self-attention. Модель сама выбирает, какие прошлые действия важнее для предсказания следующего объекта. Используется causal mask, чтобы модель не смотрела в будущее.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

BERT4Rec использует идею BERT: masked item prediction. В последовательности скрывается объект, а модель должна его восстановить.

A, B, [MASK], D -> predict masked item

SASRec обычно предсказывает следующий объект слева направо, а BERT4Rec использует двунаправленный контекст.

8.2 26. Recall@K и MRR

Recall@K показывает, какую долю релевантных объектов система нашла в первых K рекомендациях:

$$Recall@K = \frac{|Relevant\ items \cap Recommended@K|}{|Relevant\ items|}$$

Например, если пользователю релевантны 5 фильмов, а в Top-3 система нашла 2 из них, $Recall@3 = 2/5 = 0.4$.

Recall@K не учитывает точную позицию внутри Top-K. Релевантный объект на 1-м и на 10-м месте одинаково засчитывается для Recall@10.

MRR - Mean Reciprocal Rank. Он показывает, насколько высоко находится первый релевантный объект:

$$RR = \frac{1}{rank_of_first_relevant_item}$$
$$MRR = \frac{1}{|U|} \sum_{u \in U} \frac{1}{rank_u}$$

Если первый правильный объект на 1-м месте, $RR = 1$. Если на 5-м месте, $RR = 0.2$. MRR важен, когда нужен один правильный результат как можно выше.

8.3 27. DeepWalk, Node2Vec и ограничения random walks

DeepWalk строит embeddings вершин графа через случайные блуждания. Алгоритм похож на Word2Vec:

1. Запускаем random walks.
2. Получаем последовательности вершин.
3. Обучаем Skip-Gram.
4. Получаем embedding каждой вершины.

Если вершины часто встречаются рядом в прогулках, их embeddings становятся похожими.

Node2Vec улучшает DeepWalk, управляя random walk через параметры p и q. Они определяют, будет ли прогулка больше похожа на BFS или DFS.

BFS-подобное поведение помогает искать вершины из одного сообщества. DFS-подобное поведение помогает находить вершины со схожей структурной ролью.

Ограничения random walk подходов: они в основном используют структуру графа, плохо учитывают признаки объектов, имеют проблему новых вершин, зависят от качества графа и не оптимизируют recommendation metrics напрямую.

8.4 28. Message Passing, Propagation и LightGCN

Message Passing - механизм передачи информации между соседними вершинами графа. Вершина получает сообщения от соседей, агрегирует их и обновляет свой embedding.

$$m_v^{(k)} = AGGREGATE \left(\{h_u^{(k-1)} : u \in N(v)\} \right)$$
$$h_v^{(k)} = UPDATE \left(h_v^{(k-1)}, m_v^{(k)} \right)$$

В рекомендациях граф часто двудольный: пользователи связаны с объектами, с которыми взаимодействовали.

LightGCN - упрощенная графовая модель для collaborative filtering. Она убирает лишние преобразования: нет нелинейностей и обучаемых матриц в propagation. Остается распространение embeddings по user-item графу.

$$e_u^{(k+1)} = \sum_{i \in N(u)} \frac{1}{\sqrt{|N(u)| |N(i)|}} e_i^{(k)}$$
$$e_i^{(k+1)} = \sum_{u \in N(i)} \frac{1}{\sqrt{|N(i)| |N(u)|}} e_u^{(k)}$$

Финальный embedding - сумма embeddings разных слоёв:

$$e_u = \sum_{k=0}^K \alpha_k e_u^{(k)}$$
$$e_i = \sum_{k=0}^K \alpha_k e_i^{(k)}$$

Score:

$$\hat{y}_{ui} = e_u^T e_i$$

LightGCN часто обучается через BPR loss.

8.5 29. NGCF, GraphSAGE, GAT и PinSage

NGCF - Neural Graph Collaborative Filtering. Использует user-item граф и сложный message passing между пользователями и объектами. В отличие от LightGCN, NGCF применяет матрицы весов, нелинейности и взаимодействия embeddings.

GraphSAGE учится агрегировать информацию от соседей. Он хорошо подходит для больших графов, потому что может брать не всех соседей, а выборку.

$$h_v^{(k)} = \sigma \left(W \cdot CONCAT \left(h_v^{(k-1)}, AGGREGATE(\{h_u^{(k-1)}\}) \right) \right)$$

GAT - Graph Attention Network. Она использует attention, чтобы разные соседи имели разные веса.

$$h'_i = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} W h_j \right)$$

Это полезно, потому что не все действия пользователя одинаково важны.

PinSage - масштабируемая графовая модель, разработанная для Pinterest. Она использует random walks, graph convolution и выбор важных соседей, чтобы работать с огромными графами пользователей, досок и пинов.

Кратко:

NGCF - сложный graph collaborative filtering.
GraphSAGE - агрегация соседей, масштабируемость.
GAT - attention по соседям.
PinSage - GNN для огромных рекомендательных графов.

8.6 30. Attention в SASRec, BERT4Rec и GAT

Attention - механизм, который позволяет модели выбирать важные элементы. Не все действия пользователя и не все соседи графа одинаково полезны.

Общая формула self-attention:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$
$$Attention(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

В SASRec attention выбирает важные прошлые действия пользователя. Например, если пользователь смотрел ноутбук, футболку, мышку и клавиатуру, для рекомендации аксессуаров важнее ноутбук, мышка и клавиатура.

В BERT4Rec attention работает двунаправленно и помогает восстанавливать скрытые элементы последовательности. Модель использует контекст слева и справа.

В GAT attention используется на графе. Вершина получает сообщения от соседей, но каждому соседу назначается свой вес α_{ij} . Это позволяет учитывать наиболее важные связи.

Главный смысл attention в рекомендациях: модель сама учится понимать, какие прошлые действия, признаки или соседи сильнее влияют на рекомендацию.

9 КРАТКАЯ ВЕРСИЯ ДЛЯ ПОВТОРЕНИЯ

9.1 1. История рекомендательных систем

Grundy - ранняя экспертная система рекомендаций книг. Tapestry - одна из первых систем collaborative filtering. GroupLens - рекомендации новостей на основе похожих пользователей. Netflix Prize популяризировал матричную факторизацию и крупные ML-модели для рекомендаций. Эволюция: от правил к collaborative filtering, matrix factorization, нейросетям и графам.

9.2 2. Классификация и постановки задач

Content-Based рекомендует похожее по признакам. Collaborative Filtering использует поведение похожих пользователей. Hybrid объединяет подходы. Knowledge-Based использует правила. Context-Aware учитывает контекст. Prediction предсказывает оценку, Ranking сортирует объекты в Top-N.

9.3 3. Популярность, User-Based и Item-Based CF

Популярностные рекомендации показывают самые популярные объекты, но не персонализируют выдачу. IMDB использует weighted rating, чтобы учитывать и среднюю оценку, и число голосов. User-Based CF ищет похожих пользователей, Item-Based CF ищет похожие объекты.

9.4 4. Memory-Based CF

Memory-Based CF напрямую использует user-item матрицу. Система ищет похожих пользователей или объекты и строит рекомендации по их оценкам. Плюсы - простота и объяснимость. Минусы - sparse-данные, холодный старт, плохая масштабируемость.

9.5 5. Метрики сходства

Cosine similarity сравнивает направление векторов. Pearson correlation учитывает средние оценки пользователей. Jaccard сравнивает множества взаимодействий. Эти метрики нужны для поиска похожих пользователей и объектов.

9.6 6. kNN и Surprise

kNN ищет k ближайших соседей. В user-based варианте используются похожие пользователи, в item-based - похожие объекты. Surprise - Python-библиотека для классических рекомендаций: KNNBasic, KNNWithMeans, SVD, SVD++ и другие.

9.7 7. Matrix Factorization и SVD

Matrix Factorization раскладывает user-item матрицу на векторы пользователей и объектов: $R \approx PQ^T$. Предсказание: $\hat{r}_{ui} = p_u^T q_i$. Классический SVD требует заполненную матрицу, поэтому плохо подходит для sparse-рекомендаций без адаптации.

9.8 8. FunkSVD

FunkSVD обучается только на известных рейтингах. Формула: $\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$. Модель использует латентные факторы, bias и градиентный спуск. Это практичный вариант матричной факторизации для sparse-данных.

9.9 9. Pointwise, Pairwise, Listwise

Pointwise оценивает каждый объект отдельно. Pairwise сравнивает пары объектов: позитивный должен быть выше негативного. Listwise учится сортировать список целиком. Для Top-N рекомендаций pairwise и listwise обычно ближе к реальной задаче.

9.10 10. Retrieval, Ranking, Re-ranking

Retrieval быстро выбирает кандидатов из миллионов объектов. Ranking точно сортирует кандидатов. Re-ranking исправляет финальную выдачу: убирает дубли, добавляет разнообразие, учитывает бизнес-правила.

9.11 11. Content-Based Filtering

Content-Based Filtering рекомендует объекты, похожие на интересы пользователя. Профиль объекта - признаки объекта. Профиль пользователя - признаки понравившихся объектов. Utility Matrix хранит оценки и взаимодействия. Плюс - можно рекомендовать новые объекты. Минус - однообразие.

9.12 12. Bag-of-Words, TF-IDF, метаданные

Bag-of-Words считает слова в тексте. TF-IDF учитывает частоту слова и его редкость в коллекции. Метаданные - жанр, цена, категория, бренд, год и другие признаки. Всё это превращает объект в вектор для рекомендаций.

9.13 13. Cosine Similarity в content-based

Косинусная мера сравнивает профиль пользователя и объект: $\cos(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$. Чем выше сходство, тем выше объект в рекомендациях. Часто используется с TF-IDF и признаковыми векторами.

9.14 14. Гибридные системы

Гибридные системы объединяют несколько методов, например collaborative и content-based filtering. Стратегии: weighted, switching, mixed, feature combination, cascade. Они обычно точнее и лучше справляются с холодным стартом.

9.15 15. Мультимодальные данные

Мультимодальные рекомендации используют текст, изображения, видео, аудио и метаданные. Text embeddings кодируют смысл текста. Image embeddings кодируют визуальный вид объекта. Это особенно полезно для товаров, одежды, фильмов и видео.

9.16 16. LightFM

LightFM - гибридная факторизационная модель. Она использует interactions и признаки пользователей/объектов. Формула похожа на matrix factorization: $\hat{r}_{ui} = p_u^T q_i + b_u + b_i$. Хорошо помогает при холодном старте.

9.17 17. Архитектура LightFM

В LightFM embedding пользователя - сумма embeddings его признаков, embedding объекта - сумма embeddings признаков объекта. Затем считается скалярное произведение. Модель обучает не только ID, но и признаки.

9.18 18. Loss-функции LightFM

Logistic Loss используется для бинарного взаимодействия. BPR учит позитивный объект быть выше негативного. WARP улучшает верх Тор-К выдачи. k-OS WARP - более стабильная версия WARP.

9.19 19. OpenMP в LightFM

OpenMP позволяет LightFM обучаться в несколько потоков на CPU. Параметр num_threads ускоряет обучение, особенно на больших данных. Но ускорение не всегда линейное.

9.20 20. Implicit Feedback

Explicit feedback - оценки, лайки, отзывы. Implicit feedback - клики, просмотры, покупки, корзина. В implicit отсутствие действия не значит негатив. Часто используют confidence: $c_{ui} = 1 + \alpha r_{ui}$.

9.21 21. ALS

ALS раскладывает матрицу взаимодействий $R \approx XY^T$. Обучается попеременно: фиксируем item-векторы и обновляем user-векторы, потом наоборот. Для implicit feedback использует preference и confidence. Хорошо работает с большими sparse-матрицами.

9.22 22. ALS, BPR, WARP, LightFM

ALS - масштабируемая матричная факторизация. BPR - pairwise ranking. WARP - оптимизация Тор-К. LightFM - гибридная модель с признаками. Если нужны признаки - LightFM, если большая implicit-матрица - ALS, если важен порядок - BPR/WARP.

9.23 23. NCF: GMF, MLP, NeuMF

NCF заменяет простое скалярное произведение нейросетью. GMF похожа на матричную факторизацию. MLP соединяет embeddings пользователя и объекта и пропускает через нейросеть. NeuMF объединяет GMF и MLP.

9.24 24. AutoRec

AutoRec - автоэнкодер для рекомендаций. Он получает вектор оценок, сжимает его через encoder и восстанавливает через decoder. Пропущенные оценки используются как рекомендации. Бывает user-based и item-based.

9.25 25. GRU4Rec, SASRec, BERT4Rec

Последовательные модели учитывают порядок действий. GRU4Rec использует GRU и состоящие сессии. SASRec использует self-attention и causal mask. BERT4Rec использует masked item prediction и двунаправленный контекст.

9.26 26. Recall@K и MRR

Recall@K показывает, какую долю релевантных объектов система нашла в Top-K. MRR показывает, насколько высоко расположен первый релевантный объект: $RR = 1/rank$. Recall важен для полноты, MRR - для позиции первого правильного результата.

9.27 27. DeepWalk и Node2Vec

DeepWalk делает random walks по графу и обучает Word2Vec на последовательностях вершин. Node2Vec управляет блужданиями через параметры p и q , приближая их к BFS или DFS. Минусы: холодный старт, слабое использование признаков, не оптимизирует ranking напрямую.

9.28 28. Message Passing и LightGCN

Message Passing - передача информации между соседями графа. LightGCN упрощает GCN: убирает нелинейности и матрицы весов, оставляет propagation embeddings по user-item графу. Score считается как $e_u^T e_i$.

9.29 29. NGCF, GraphSAGE, GAT, PinSage

NGCF - графовая collaborative filtering модель со сложным propagation. GraphSAGE агрегирует соседей и масштабируется на большие графы. GAT использует attention по соседям. PinSage применяет random walks и graph convolution для огромных графов Pinterest.

9.30 30. Attention в рекомендациях

Attention помогает выбирать важные элементы. В SASRec он выбирает важные прошлые действия. В BERT4Rec помогает восстанавливать скрытые элементы последовательности. В GAT выбирает важных соседей в графе. Главная идея: не все действия и связи одинаково важны.

10 САМЫЙ КОРОТКИЙ ИТОГ

Рекомендательные системы помогают выбрать подходящие объекты из большого множества. Простые методы рекомендуют популярное. Collaborative filtering использует поведение пользователей. Content-based filtering использует признаки объектов. Matrix factorization строит скрытые векторы пользователей и объектов. LightFM добавляет признаки к факторизации. ALS, BPR и WARP работают с implicit feedback и ранжированием. Нейросетевые модели вроде NCF и AutoRec учат сложные зависимости. Последовательные модели учитывают порядок действий. Графовые модели используют связи пользователь-объект. Attention помогает выбрать самые важные действия или связи.